

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 5: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper functions live in `starter_code.py` at the week root: `generate_weighted_graph`, `empty_graph`, `add_edge`, `edge_list`, `vertices`, and the `UnionFind` class (with path compression and union by rank). A graph is an *adjacency dict*: a map from each vertex to a dict `{neighbour: weight}`; undirected edges appear in both endpoints' dicts.

The 12 problems are grouped into three themes:

- **Shortest Paths** (Problems 1–6): a weighted-graph builder, Dijkstra's algorithm (distances and path reconstruction), Bellman–Ford with negative edges, cross-validation, and a counter-example showing Dijkstra fails on a negative edge.
- **Minimum Spanning Trees** (Problems 7–11): Kruskal's algorithm with union-find, Prim's algorithm with a heap, verification that they agree, and the cut and cycle properties.
- **Applications** (Problem 12): single-link k -clustering via Kruskal.

1 Shortest Paths

Problem 1 – Weighted Graph Builder and Edge Listing

Implement `build_graph(edges, directed=False)` that builds an adjacency-dict graph from a list of `(u, v, w)` edge tuples (using `add_edge` from `starter_code.py`), `neighbors(graph, u)` returning `u`'s neighbour→weight dict (empty if isolated), `list_edges(graph, directed=False)` returning each edge once as `(u, v, w)`, and `total_weight(graph, directed=False)` summing all edge weights.

Example. `build_graph([(0,1,4),(1,2,3),(0,2,5)])` yields a graph with `g[0][1] == 4` and `g[1][0] == 4`; `total_weight` is 12.

Problem 2 – Dijkstra's Shortest-Path Distances

Implement `dijkstra(graph, source)` using a binary heap (`heapq`) to return a dict of shortest-path distances from `source` to every vertex (non-negative weights assumed; unreachable vertices get `float('inf')`).

Hint. Use the lazy heap version: push `(distance, vertex)` on each improvement and skip a popped vertex that has already been finalized.

Problem 3 – Dijkstra with Predecessors and Path Reconstruction

Implement `dijkstra_with_pred(graph, source)` returning `(dist, pred)`, where `pred[v]` is the vertex preceding `v` on a shortest path (`None` for the source and for unreachable vertices). Then implement `reconstruct_path(pred, source, target)` returning the vertex list `[source, ..., target]`, or `None` if `target` is unreachable.

Problem 4 – Bellman–Ford with Negative Edges

Implement `bellman_ford(graph, source, directed=False)`: relax all edges $n - 1$ times, then do one extra pass to detect a negative cycle. Return the distance dict, or `None` if a negative cycle is detected. Unlike Dijkstra, this handles negative weights.

Note. For an undirected graph, relax each edge in both directions (and any negative undirected edge is itself a negative cycle).

Problem 5 – Verifying Dijkstra and Bellman–Ford Agree

Implement `agree_on_random_graph(n, seed)`: generate a random connected non-negative graph (`generate_weighted_graph(n, seed=seed, edge_prob=0.4, max_weight=15)`) and return `True` iff `dijkstra` (Problem 2) and `bellman_ford` (Problem 4) give equal distances from source 0. Then implement `verify_agreement(num_trials, n, seed)` that returns `True` iff this holds on `num_trials` random instances (using `seed + trial`).

Problem 6 – Counter-Example: Dijkstra Fails on a Negative Edge

Implement `negative_edge_graph()` returning a small directed graph with one negative edge (and no negative cycle) on which Dijkstra is wrong. Then implement `dijkstra_vs_bellman_ford()` returning `(dijkstra_dist, bellman_ford_dist)` for that graph; the two dicts must differ at some vertex.

Hint. Make Dijkstra *finalize* a vertex at its direct-edge distance before it ever explores the cheaper route that ends with the negative edge.

2 Minimum Spanning Trees

Problem 7 – Kruskal’s MST with Union-Find

Implement `kruskal(graph)`: sort the edges by weight and add each edge whose endpoints lie in different components (tested with the `UnionFind` class). Return `(mst_edges, total_weight)`.

Example. On a 4-vertex connected graph, the MST has exactly 3 edges; for the graph in the starter tests its total weight is 6.

Problem 8 – Prim’s MST with a Heap

Implement `prim(graph, start=None)`: grow a tree from `start` (default: the first vertex), always adding the cheapest edge crossing from the tree to a vertex outside it, using a binary heap. Return the total MST weight. Assume the graph is connected.

Note. The MST total weight is independent of the start vertex.

Problem 9 – Verifying Kruskal and Prim Agree

Implement `kruskal_equals_prim(n, seed)`: generate a random connected graph and return `True` iff `kruskal` (Problem 7) and `prim` (Problem 8) give the same total MST weight. Then implement `verify_mst_agreement(num_trials, n, seed)` returning `True` iff this holds on every random trial. (They may pick different edge *sets* when weights tie, but the total must match.)

Problem 10 – Verifying the Cut Property

Implement `crossing_edges(graph, S)` returning all edges with exactly one endpoint in the set `S`, `min_crossing_edge(graph, S)` returning a minimum-weight crossing edge (or `None`), and `cut_property_holds(graph, S)` verifying that the minimum crossing edge of the cut $(S, V \setminus S)$ belongs to some MST (build an MST via Kruskal; in the tie case, force the edge in and confirm the total weight is unchanged).

Problem 11 – Classify an Edge: in EVERY / SOME / NO MST

Implement `classify_edge(graph, edge)` returning the string `"every"`, `"some"`, or `"none"` according to whether `edge` is in every, some (but not all), or no MST. Compare the global MST weight with the MST weight when the edge is *forced in* and when it is *forbidden*: forcing it worsens the weight $\Rightarrow$ `"none"`; forbidding it worsens the weight $\Rightarrow$ `"every"`; otherwise `"some"`.

Hint. This is the computational form of the cut and cycle properties.

3 Applications

Problem 12 – Single-Link k -Clustering via Kruskal

Implement `single_link_clustering(graph, k)`: run Kruskal's algorithm but stop once exactly k components remain. Return `(clusters, spacing)`, where `clusters` is a list of k lists of vertices and `spacing` is the minimum inter-cluster edge weight (the next edge Kruskal would have added), or `float('inf')` when k equals the number of vertices.

Example. For points with cheap edges 0–1 and 2–3 (weight 1) and an expensive bridge 1–2 (weight 10), the 2-clustering is $\{0, 1\}$ and $\{2, 3\}$ with spacing 10.

4 LeetCode Practice

After completing the problems above, practise this week's techniques (Dijkstra, Bellman–Ford, and minimum spanning trees) on the following [LeetCode](#) problems. Difficulty is LeetCode's own label.

- [Network Delay Time](#) – *Medium* – Dijkstra.
- [Path with Minimum Effort](#) – *Medium* – Dijkstra / binary search.
- [Cheapest Flights Within K Stops](#) – *Medium* – Dijkstra / Bellman–Ford.
- [Swim in Rising Water](#) – *Hard* – Dijkstra / union-find.
- [Path With Maximum Probability](#) – *Medium* – Dijkstra variant.
- [Min Cost to Connect All Points](#) – *Medium* – MST (Prim / Kruskal).
- [Connecting Cities With Minimum Cost](#) – *Medium* – MST (Kruskal).
- [Find the City With the Smallest Number of Neighbors at a Threshold Distance](#) – *Medium* – shortest paths.
- [Reconstruct Itinerary](#) – *Hard* – greedy / Hierholzer.